COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	9/10/2026	**DATE SUBMITTED:**	9/15/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 8 **Sorting Algorithms 2**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 8-1. Array of Values for Sort Algorithm Testing

CODE

```cpp
#include <iostream>
#include <random>
#include "sorting2.h"
void displayArray(int arr[], int size);
const int size = 100;

int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 number( sd: rd());
    std::uniform_int_distribution<int> dist( a: 1,  b: 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dist( [&] number);
    }

    std::cout << "\nUnsorted Array\n";
    displayArray(dataset, size);
```

OUTPUT

```
Unsorted Array
41 32 99 77 93 56 54 99 53 55 94 10 91 31 27 23 72 47 13 37 80 87 48 1 29 5 42 58 37 81 81 13 95 14 49 100 66 46 76 24 4
 57 59 69 17 60 28 88 41 98 40 74 49 47 11 39 93 66 67 69 87 99 100 8 57 3 87 22 51 30 74 31 71 83 84 27 51 64 35 88 93
64 30 31 73 67 80 42 64 62 48 59 57 78 44 41 98 47 58 4
```

OBSERVATION

So I actually used the standard library “random” because it is much more equal in the randomness compared to the modulo method. The output seen is not consistent because the seed is based on the “std::random_device” syntax that generates a totally random number every time the program runs. The line after that is the “std::mt19937” that is the actual random number engine that is known for its pseudo-random number generation algorithm. This is with the variable name “number”. The “std::uniform_int_distribution<int>” is where we can put a range on the random generation of numbers. In this case we call the variable “dist”, and the values 1 and 100 passed defines the range “dist” will produce. So by calling “dist” with “number”, the engine produces a raw pseudo-random value and “dist” just takes that value and shapes it with no bias to the 1-100 range.

Table 8-2. Shell Sort Technique

CODE

```
int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 number( sd: rd());
    std::uniform_int_distribution<int> dist( a: 1,  b: 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dist( &: number);
    }

    std::cout << "\nUnsorted Array\n";
    displayArray(dataset, size);


    std::cout << "\nShell Sorted Array\n";
    shellSort(dataset, size);
    displayArray(dataset, size);
    return 0;
}
```

```cpp
template <typename T>
void shellSort(T arr[], int size) {
    /*
    shellSort(array, size)
        for interval i <- size/2n down to 1
            for each interval "i" in array
                sort all the elements at interval "i"
    end shellSort
    */

    for (int gap = size/2 ; gap >= 1; gap = gap/2) {
        for (int k = gap ; k < size; k++ ) {
            int temp = arr[k];
            int pos = k;
            while (pos >= gap && arr[pos - gap] > temp) {
                arr[pos] = arr[pos - gap];
                pos = pos - gap;
            }
            arr[pos] = temp;
        }
    }
}
```

OUTPUT

```
Unsorted Array
19 35 22 28 38 5 19 9 64 44 46 41 2 40 32 1 60 28 32 4 31 74 80 18 80 87 6 64 23 19 47 42 21 30 27 2 51 96 100 63 59 2 9
4 50 62 63 76 70 35 89 23 77 24 49 9 30 21 48 56 6 70 51 96 94 89 83 93 31 65 26 51 29 55 61 77 2 40 82 34 41 100 9 50 9
2 92 69 1 18 56 73 47 28 32 57 42 25 1 69 82 42
Shell Sorted Array
1 1 1 2 2 2 2 4 5 6 6 9 9 9 18 18 19 19 19 21 21 22 23 23 24 25 26 27 28 28 28 29 30 30 31 31 32 32 32 34 35 35 38 40 40
 41 41 42 42 42 44 46 47 47 48 49 50 50 51 51 51 55 56 56 57 59 60 61 62 63 63 64 64 65 69 69 70 70 73 74 76 77 77 80 80
 82 82 83 87 89 89 92 92 93 94 94 96 96 100 100
```

OBSERVATION
The function worked and successfully sorted the unsorted array. Shell Sort (in my case) uses nested loops (3 loops in total). The outer loop is what determines the gap, and it will stop when the gap is less than 1. With every increment, the gap divides by 2. The first inner loop indicates where we will start the actual sorting process. This loop only stops when k is equal to or greater than the size. Inside this first inner loop, we assign "temp" as our number to check, and we keep note of our initial position in the array with the variable "pos". Now we have the second inner loop which keeps running as long as "pos" is still greater than or equal to the gap, and the value at "arr[pos - gap]" is greater than "temp". If either of those conditions becomes false, the loop stops. Inside the loop, if "arr[pos - gap]" is greater than "temp", that value gets shifted forward into "arr[pos]" (pushing the bigger value into the position we're vacating). After that, "pos" is reassigned to "pos - gap", moving our position backward by one gap-group step. Once the while loop stops, "pos" now holds the correct empty slot, so we assign "temp" into "arr[pos]", effectively switching both of the elements' places. The best case time complexity for the Shell sort is O(n log n) while the worst case time complexity would be O(n^2)

Table 8-3. Merge Sort Algorithm

CODE

```
int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 number( sd: rd());
    std::uniform_int_distribution<int> dist( a: 1,  b: 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dist( &: number);
    }

    std::cout << "\nUnsorted Array\n";
    displayArray(dataset, size);


    std::cout << "\nMerge Sorted Array\n";
    mergeSort(dataset,  left: 0,  right: size - 1);
    displayArray(dataset, size);

    return 0;
}
```

```cpp
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    T temp[right - left + 1];

    int i = left;
    int j = middle + 1;
    int temp_pos = 0;

    while (i <= middle && j <= right) {
        if (arr[i] < arr[j]) {
            temp[temp_pos] = arr[i];
            i++;
            temp_pos++;
        } else {
            temp[temp_pos] = arr[j];
            j++;
            temp_pos++;
        }
    }
    while (i <= middle) {
        temp[temp_pos] = arr[i];
        i++;
        temp_pos++;
    }
    while (j <= right) {
        temp[temp_pos] = arr[j];
        j++;
        temp_pos++;
    }
    int k = 0;
    for (int i = left; i <= right; i++) {
        arr[i] = temp[k];
        k++;
    }
}
```

```cpp
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left >= right) {
        return;
    }

    int middle = (left + right) / 2;

    mergeSort(arr, left, right: middle);
    mergeSort(arr, left: middle + 1, right);
    merge(arr, left, middle, right);
}
```

OUTPUT

```
"D:\-COLLEGE\SECOND YEAR\CODING\HOA 8\cmake-build-debug\HOA_8.exe"

Unsorted Array
19 91 96 86 56 37 43 18 71 60 5 13 20 8 96 53 58 70 42 14 45 28 20 84 75 33 50 42 8 47 84 79 50 96 59 36 19 15 93 40 65
91 55 75 59 65 47 95 93 76 27 67 98 38 80 84 4 59 24 59 60 41 95 72 86 11 84 54 53 1 18 80 78 25 99 45 41 50 81 51 27 80
 36 17 1 9 55 47 29 81 9 51 98 78 77 9 55 68 89 11
Merge Sorted Array
1 1 4 5 8 8 9 9 9 11 11 13 14 15 17 18 18 19 19 20 20 24 25 27 27 28 29 33 36 36 37 38 40 41 41 42 42 43 45 45 47 47 47
50 50 50 51 51 53 53 54 55 55 55 56 58 59 59 59 59 60 60 65 65 67 68 70 71 72 75 75 76 77 78 78 79 80 80 80 81 81 84 84
84 84 86 86 89 91 91 93 93 95 95 96 96 96 98 98 99
Process finished with exit code 0
```

OBSERVATION

For this number, we used the Merge sorting technique, it uses the “divide and conquer” strategy. My observation is that the function “mergeSort” is recursive. It calls it 2 times, the first time for the left side of the array and the second time for the right side. This function is dedicated to only dividing, the real sorting happens when we call the “merge” function. The idea here is that it takes the left side’s first value (i) and the right side’s first value (j). If one value is greater than the other, we will increment the respective variable (i or j), and put that in a temporary array where the “soon-to-be” sorted elements will be held. This will loop until every element has been accounted for, and this will happen for both the left and right side of the original array until we’re back to one fully sorted array with the original size. The best and worst case time complexity of Merge sort is O(n log n).

Table 8-4. Quick Sort Algorithm

CODE

```
int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 number( sd: rd());
    std::uniform_int_distribution<int> dist( a: 1,  b: 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dist( &number);
    }

    std::cout << "\nUnsorted Array\n";
    displayArray(dataset, size);

    std::cout << "\nQuick Sorted Array\n";
    quickSort(dataset,  low: 0,  high: size);
    displayArray(dataset, size);

    return 0;
}
```

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot_value = arr[high];
    int boundary = low - 1;

    for (int i = low; i <= high-1; i++) {
        if (arr[i] < pivot_value) {
            boundary++;
            std::swap( a: [&]arr[i], b: [&]arr[boundary]);
        }
    }
    std::swap( a: [&]arr[high], b: [&]arr[boundary+1]);
    return boundary+1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    /*
        if(low < high)
        pivot = partition (A,low,high);
        quicksort(A,low,pivot-1)
        quicksort(A,pivot+1,high)
    */

    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, high: pivot-1);
        quickSort(arr, low: pivot + 1, high);
    }
}
```

OUTPUT

```
Unsorted Array
66 54 7 95 14 51 54 39 24 49 27 31 23 99 85 6 2 68 13 33 69 51 95 15 48 59 21 11 10 66 95 29 71 29 52 33 30 97 32 74 95
94 10 59 39 56 34 31 24 26 72 23 54 7 14 7 67 99 22 12 83 40 69 52 29 94 96 84 89 3 6 93 10 1 18 29 50 63 70 39 23 88 86
 98 23 11 51 60 19 30 27 85 83 26 92 21 14 47 70 19
Quick Sorted Array
1 2 3 6 6 7 7 7 10 10 10 11 11 12 13 14 14 14 15 18 19 19 21 21 22 23 23 23 23 24 24 26 26 27 27 29 29 29 29 30 30 31 31
 32 33 33 34 39 39 39 40 47 48 49 50 51 51 51 52 52 54 54 54 56 59 59 60 63 66 66 67 68 69 69 70 70 71 72 74 83 83 84 85
 85 86 88 89 92 93 94 94 95 95 95 95 96 97 98 99 99
Process finished with exit code 0
```

OBSERVATION
For the last activity in the procedures, we utilized the quick sorting technique. This uses a concept called “partitioning”. It uses a “pivot” as a reference point to rearrange the values. In the program, we have two functions that make this happen, “quickSort” and “partition”. The first function is responsible for assigning the pivot for the second function to work with. The “partition” function works like this, it assigns the pivot in an array (in this case, the very last element), and has two tracking indices, i and boundary. The i traverses the list from the start, looking for values smaller than the pivot. The boundary increments when i finds a value, and then swaps those. This is done until i has reached the pivot point, and the last swap happens before partitioning again. This is done until the array has been fully sorted. The best case time complexity for Quick sort is O(n log n) and the worst case time complexity is O(n^2).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, you can. Here is a demonstration where merge sort is called twice rather than quick sort

CODE

```
int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 number( sd: rd());
    std::uniform_int_distribution<int> dist( a: 1,  b: 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dist( [&] number);
    }

    std::cout << "\n\nQuick Sorted Array\n";
    quickSort(dataset,  low: 0,  high: size);
    displayArray(dataset, size);

    return 0;
}
```

```cpp
#ifndef SUP_ACT_8_SORTINGSUPP_H
#define SUP_ACT_8_SORTINGSUPP_H
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    T temp[right - left + 1];

    int i = left;
    int j = middle + 1;
    int temp_pos = 0;

    while (i <= middle && j <= right) {
        if (arr[i] < arr[j]) {
            temp[temp_pos] = arr[i];
            i++;
            temp_pos++;
        } else {
            temp[temp_pos] = arr[j];
            j++;
            temp_pos++;
        }
    }
    while (i <= middle) {
        temp[temp_pos] = arr[i];
        i++;
        temp_pos++;
    }
    while (j <= right) {
        temp[temp_pos] = arr[j];
        j++;
        temp_pos++;
    }
    int k = 0;
    for (int i = left; i <= right; i++) {
        arr[i] = temp[k];
        k++;
    }
}
```

```cpp
template <typename T>
void mergeSort(T arr[], int left, int right) {

    /*
    If N=1, Arr is already sorted
    If N>1,
    Left = 0, right = N-1
    Find middle = (left + right)/2
    Call merge_sort(Arr,left,middle) =>sort first half recursively
    Call merge_sort(Arr,middle+1,right) => sort second half recursively
    Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
    Exit
    */

    if (left >= right) {
        return;
    }
    int middle = (left + right) / 2;

    mergeSort(arr, left, right: middle);
    mergeSort(arr, left: middle + 1, right);
    merge(arr, left, middle, right);
}
```

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot_value = arr[high];
    int boundary = low - 1;

    for (int i = low; i <= high-1; i++) {
        if (arr[i] < pivot_value) {
            boundary++;
            std::swap( a: [&]arr[i], b: [&]arr[boundary]);
        }
    }
    std::swap( a: [&]arr[high], b: [&]arr[boundary+1]);
    return boundary+1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    /*
        if(low < high)
        pivot = partition (A,low,high);
        quicksort(A,low,pivot-1)
        quicksort(A,pivot+1,high)
    */

    if (low < high) {
        int pivot = partition(arr, low, high);
        mergeSort(arr, left: low, right: pivot-1);
        mergeSort(arr, left: pivot+1, right: high);
    }
}
```

OUTPUT

```
Quick Sorted Array
4 7 10 11 12 13 16 16 16 17 20 20 22 23 23 25 27 28 29 29 30 31 31 32 33 34 34 34 35 35 35 35 36 37 38 39 41 43 44 45 46
 46 49 49 51 51 52 52 53 54 55 55 55 55 59 60 60 60 63 64 65 66 66 67 68 68 69 71 71 74 74 75 76 76 76 78 81 81 82 83 83
 84 85 85 85 86 86 86 88 88 90 91 93 95 97 98 99 100 100 100
Process finished with exit code 0
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- Merge Sort / Quick sort would give me the fastest time performance.
- The reason why merge and quick sort both have O(n log n) for their time complexity is because they use the same "divide and conquer" strategy. Both of them split the array into sub-arrays to make the sorting and swapping easier.

CONCLUSION

What I've learned in this HOA activity is the concept of Shell, Merge, and Quick Sorting techniques. For the procedure, we've done Shell Sort. To put into simple terms, it is a modified insertion sort that has a "gap" which equals the size divided by 2. This gap gets divided by 2 each pass until it reaches 1. The Merge Sort uses a "divide and conquer" strategy. It divides the array into sub-arrays until it's only 1. At this point, it merges two elements in sorted order, then, two pairs, then two groups of four, and so on until we reach the original size of the array. For the last activity in procedure, Quick sort uses the same strategy as Merge Sort, but with a new concept called "partitioning". It uses a "pivot" as a reference point in sorting the array and sub-arrays. In my case, my "pivot" is always the last element. After sorting, it partitions it and picks a new pivot point. This will repeat until everything gets sorted.

The supplementary activity, for problem 1. Instead of recursively calling quicksort, we can call merge sort instead to sort the partitions. Showing that it's possible to use different sorting algorithms for quick sort. For problem 2, I've already explained it, but in simpler terms; the reason why merge and quick sort have a O(n log n) time complexity is because they both use the same "divide and conquer" strategy.

In conclusion, I did fairly good in this HOA. I need to improve how I visualize my code and how I interpret it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts and Sequence of the Sorting Algorithms**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It showed visualization of how the Sorting Algorithms work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.